

0.78. Le rebond : une vitesse qui change de signe

Une lettre qui va et vient d'un bord à l'autre, toute seule : $x = x + vx$, et au bord, vx change de signe.

1. Ce qu'il faut comprendre

Une balle qui rebondit : la lettre O avance vers la droite ; au bord droit, elle repart vers la gauche ; au bord gauche, vers la droite. Sans fin, et sans compter les pas.

Ce qui est nouveau ici : une VITESSE, vx . Elle dit de combien bouge la lettre à chaque pas : 1 , une case à droite ; -1 , une case à gauche. À chaque pas, $x = x + vx$;

Le rebond : au bord droit, $vx = -1$; au bord gauche, $vx = 1$;. On ne change **pas** la position, seulement la **direction** : le pas suivant se fait dans l'autre sens.

-1 dans un octet : il est rangé 255. $x + 255$ « tourne » et revient à $x - 1$: c'est pour cela que $x = x + vx$; recule quand vx vaut -1 .

Le rythme, avec `chaque(50)` (le 0.77) : un pas toutes les 50 ms, 20 cases par seconde.

C'est la base des jeux de balle : Pong, casse-briques. Le 0.78.1 fait rebondir sur les deux axes.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <chaque>` → [0.29.3. La fonction `chaque\(\)` — toutes les n millisecondes](#)
- `#include <effacer>` → [0.16.3. La fonction `effacer\(\)` — effacer des cases](#)
- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **405 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	56 o	← la console, à l'allumage — la mise en route de la console
chaque_minuteur()	124 o	← <code>main()</code> — une fonction fournie par le langage
<code>main()</code>	78 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← <code>EffacerCases</code> — attend de pouvoir écrire une case
AdresseCase	10 o	← <code>effacer()</code> , <code>poser()</code> — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← <code>effacer()</code> — efface des cases
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran

```

    Tuiles          16 o ← CopierTuiles - 1 lettre sur 43
Lettres gravées : 0
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
    effacer()      × 1    15 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
    poser()        × 1    13 o sur place, dans main() + AdresseCase (10 o)
    image()        × 1     3 o sur place, dans main() + AttendreImage (28 o)
    images()       × 1     2 o sur place, dans chaque_minuteur()
Tes fonctions appelées :
    chaque_minuteur() × 1    10 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <chaque>      // répond 1 toutes les n millisecondes
#include <effacer>     // efface des cases, ou tout le fond
#include <poser>       // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A

```

2. UNE VARIABLE — X

```

uint8_t x = 0;        // la colonne de la balle

```

3. UNE VARIABLE — VX

```

uint8_t vx = 1;       // sa VITESSE : 1 = vers la droite, -1 = vers la gauche

```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    while (true) {
        image();
        if (chaque(50)) {                // 20 fois par seconde
            effacer(x, 5, 1);             // 1. efface l'ancienne place
            x = x + vx;                    // 2. avance de vx : +1 ou -1

            // 3. Le rebond : au bord, la vitesse change de signe.
            //
            //  0  1  2 ... 18 19
            //  0 → → → → → 0      au bord droit (19) : vx = -1, elle repart à gauche
            //  0 ← ← ← ← ← 0      au bord gauche (0) : vx = 1, elle repart à droite
            if (x == 19) {
                vx = -1;
            }
            if (x == 0) {
                vx = 1;
            }

            poser(x, 5, ALPHABET[14]);     // 4. la balle : ALPHABET[14], le O
        }
    }
}

```

3. Ce qui se passe, étape par étape

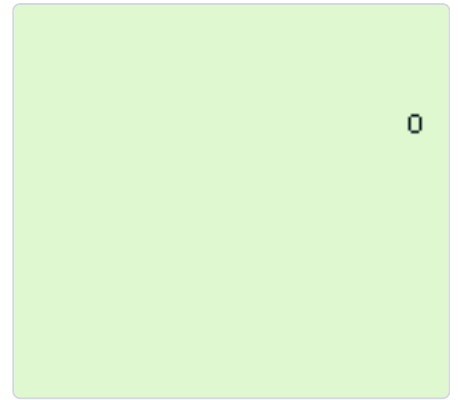
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



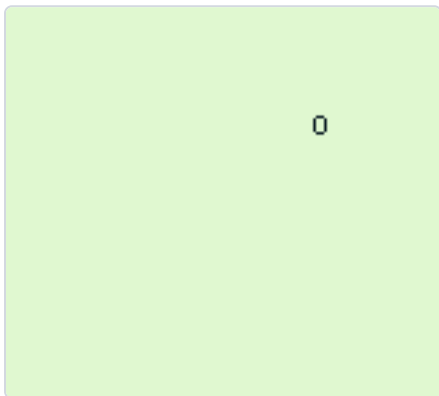
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un O qui file d'un bord à l'autre sur la ligne 5, et rebondit sans fin.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la balle touche le bord droit (19)
- ✓ et repart à gauche
- ✓ le O est à sa place

Ensuite : **0.78.1. Le rebond : une vitesse qui change de signe — en diagonale** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++